

Chapter 1: Introduction to Compiling

1. Compilers

- A compiler translates high-level source code (e.g., C, Java) into low-level machine code or intermediate code.
- Key tasks: **Translation** (source to target code) and **Error Detection** (syntax, semantic errors).

2. Analysis of the Source Program

- Compiler performs **analysis** (understanding the source code) and **synthesis** (generating target code).
- Analysis includes lexical, syntax, and semantic analysis.

3. Phases of a Compiler

- **Lexical Analysis**: Converts characters into tokens (e.g., `int` → token).
- **Syntax Analysis**: Parses tokens into a syntax tree (e.g., checking grammar rules).
- **Semantic Analysis**: Checks meaning (e.g., type checking).
- **Intermediate Code Generation**: Generates abstract intermediate code.
- **Code Optimization**: Improves efficiency of the code.
- **Code Generation**: Converts intermediate code to machine code.

4. Cousins of the Compiler

- **Preprocessor**: Handles macros, file inclusion.
- **Assembler**: Converts assembly code to machine code.
- **Linker**: Combines object files into an executable.
- **Loader**: Loads the executable into memory.

5. Grouping of Phases

- **Front-end**: Analysis (lexical, syntax, semantic).
- **Back-end**: Synthesis (optimization, code generation).

6. Compiler-Construction Tools

- **Parser Generators**: Create parsers from grammars (e.g., YACC, Bison).
- **Scanner Generators**: Create lexical analyzers (e.g., Lex, Flex).
- **Syntax-directed Translation Engines**: Generate intermediate code.

Chapter 4: Syntax Analysis

1. Role of the Parser

- Parses tokens into a **parse tree** or **syntax tree**.
- Ensures the program follows the grammar rules.

2. Context-Free Grammars (CFG)

- A formal grammar used to describe syntax.
- Components: **Terminals**, **Non-terminals**, **Productions**, **Start Symbol**.

3. Writing a Grammar

- Define rules for valid constructs (e.g., $E \rightarrow E + T \mid T$).
- Avoid ambiguity and left recursion.

4. Top-Down Parsing

- Starts from the start symbol and derives the input string.
- Methods: **Recursive Descent Parsing**, **LL(1) Parsing**.
- Issues: Left recursion, backtracking.

5. Bottom-Up Parsing

- Starts from the input string and reduces it to the start symbol.
- Methods: **Shift-Reduce Parsing**, **LR Parsing**.

6. LR Parsers

- Efficient bottom-up parsers (e.g., LR(0), SLR(1), LALR(1), CLR(1)).
- Use parsing tables to decide actions (shift, reduce, accept, error).

7. Using Ambiguous Grammars

- Ambiguous grammars can be used with **disambiguation rules** (e.g., operator precedence).

8. Parser Generators

- Tools like **YACC** or **Bison** generate parsers from CFGs.
-

Chapter 5: Syntax-Directed Translation

1. Syntax-Directed Definitions (SDD)

- Associate semantic rules with grammar productions.
- Two types: **S-attributed** (synthesized attributes) and **L-attributed** (inherited attributes).

2. Construction of Syntax Trees

- Builds a tree representation of the program.
- Nodes represent constructs (e.g., expressions, statements).

3. Bottom-Up Evaluation of S-Attributed Definitions

- S-attributed definitions can be evaluated during bottom-up parsing.
- Attributes are computed from children to parent.

4. L-Attributed Definitions

- Attributes depend on siblings or parent nodes.
- Can be evaluated in top-down or bottom-up parsers.

5. Top-Down Translation

- Attributes are evaluated during top-down parsing.
- Requires L-attributed definitions.

6. Bottom-Up Evaluation of Inherited Attributes

- Inherited attributes are passed down the parse tree.
- Requires special techniques (e.g., marker non-terminals).

7. Recursive Evaluators

- Recursive functions evaluate attributes during parsing.

8. Analysis of Syntax-Directed Definitions

- Check for circular dependencies and ensure attributes can be computed.
-

Key Points to Remember

- **Compiler Phases:** Lexical → Syntax → Semantic → Intermediate Code → Optimization → Code Generation.
- **Parsing:** Top-down (LL) vs. Bottom-up (LR).
- **Syntax-Directed Translation:** S-attributed (bottom-up) vs. L-attributed (top-down).
- **Tools:** Lex, YACC, Bison for automating compiler construction.

Chapter 1: Introduction to Compiling

1.1 Compilers

A compiler is a program that translates a high-level programming language into machine code or an intermediate representation. It enables programmers to write code in human-readable formats and converts it into an executable form.

1.2 Analysis of the Source Program

A compiler performs analysis to understand the structure and semantics of the source code. This involves:

- **Lexical Analysis:** Tokenizing the source code.
- **Syntax Analysis:** Checking the grammatical structure.
- **Semantic Analysis:** Ensuring logical correctness.

1.3 The Phases of a Compiler

A compiler consists of multiple phases, grouped into two parts: **Analysis (Front-end)** and **Synthesis (Back-end)**.

1. **Lexical Analysis:** Converts input into tokens.
2. **Syntax Analysis:** Parses tokens into a parse tree.
3. **Semantic Analysis:** Ensures meaningfulness of code.
4. **Intermediate Code Generation:** Converts into an intermediate representation.
5. **Code Optimization:** Improves performance.
6. **Code Generation:** Produces target machine code.
7. **Symbol Table Management & Error Handling** are integral throughout.

1.4 Cousins of the Compiler

Other related tools include:

- **Preprocessors:** Handle macros and directives.
- **Assemblers:** Convert assembly language into machine code.
- **Linkers and Loaders:** Resolve addresses and load programs into memory.
- **Interpreters:** Execute source code line by line.

1.5 The Grouping of Phases

Compiler phases are grouped as:

- **Front-end:** Lexical, Syntax, Semantic Analysis.
- **Middle-end:** Intermediate Code Generation and Optimization.
- **Back-end:** Code Generation and Optimization.

1.6 Compiler-Construction Tools

Tools that assist in compiler development:

- **Lexical Analyzers (Lex/Flex)**
 - **Parser Generators (YACC/Bison)**
 - **Syntax-Directed Translation Engines**
 - **Code Optimization Tools**
 - **Debugging and Profiling Tools**
-

Chapter 4: Syntax Analysis

4.1 The Role of the Parser

The parser ensures that the given input program follows the correct syntax by constructing a parse tree and detecting errors.

4.2 Context-Free Grammars (CFG)

A CFG consists of:

- **Terminals:** Basic symbols (e.g., keywords, operators).
- **Non-Terminals:** Variables representing language constructs.
- **Production Rules:** Define how non-terminals are replaced.
- **Start Symbol:** The initial rule for parsing.

4.3 Writing a Grammar

Guidelines for grammar design:

- Must be unambiguous.
- Should be easy to parse.
- Must capture the syntactic structure of the language.

4.4 Top-Down Parsing

Parses input from left to right, constructing the parse tree from the top.

- **Recursive Descent Parsing:** Uses recursive procedures.
- **Predictive Parsing (LL Parsing):** Uses a lookahead pointer to decide the correct production.

4.5 Bottom-Up Parsing

Constructs a parse tree from leaves to the root.

- **Shift-Reduce Parsing:** Uses a stack.
- **Operator-Precedence Parsing:** Works on operator precedence.

4.7 LR Parsers

LR parsers (Left-to-right, Rightmost derivation):

- **LR(0), SLR(1), LALR(1), and CLR(1)** are variations.
- More powerful than LL parsers.

4.8 Using Ambiguous Grammars

Ambiguous grammars lead to multiple parse trees. Resolution methods include:

- Operator precedence.
- Associativity rules.
- Grammar rewriting.

4.9 Parser Generators

Tools like **YACC (Yet Another Compiler Compiler)** automate parser creation.

Chapter 5: Syntax-Directed Translation

5.1 Syntax-Directed Definitions (SDD)

An SDD attaches semantic rules to grammar productions to guide translation.

5.2 Construction of Syntax Trees

Syntax trees represent the hierarchical structure of code. Nodes represent operations, and leaves represent operands.

5.3 Bottom-Up Evaluation of S-Attributed Definitions

- Uses **S-attributed definitions**, which rely on synthesized attributes only.
- Attributes are computed during parsing.

5.4 L-Attributed Definitions

- Uses both synthesized and inherited attributes.
- Values are passed in a left-to-right order.

5.5 Top-Down Translation

Translation occurs simultaneously with parsing, using inherited attributes.

5.6 Bottom-Up Evaluation of Inherited Attributes

Inherited attributes pass values from parent nodes to children in a bottom-up manner.

5.7 Recursive Evaluators

Recursive traversal of the parse tree for attribute computation.

5.10 Analysis of Syntax-Directed Definitions

- Complexity depends on attribute dependencies.
- Translation is easier when attributes do not require multiple passes.